Introduction

Every Python developer uses dictionaries daily, but very few can explain what actually happens inside one when two different-looking keys turn out to be the same key underneath. That gap in understanding is exactly what interviewers like to probe, because it separates people who memorized dict syntax from people who understand how dict *works*.

This is one of those small, sharp examples: three lines of code that look completely predictable until you actually run them. Understanding why `1` and `True` collide as dictionary keys teaches you something bigger than a party trick — it teaches you how Python defines equality, how that definition drives hashing, and how a hash table decides whether to store a new entry or silently merge it into an existing one. That's core computer science, and it shows up constantly in interviews that touch data structures.

Problem Overview

Here's the setup, in plain terms: you create an empty dictionary. You set the key `1` to the value `"one"`. Then you set the key `True` to the value `"yes"`. Finally, you print the dictionary.

The question is simple to state and hard to guess correctly on the first try: what does the dictionary contain after both assignments? Does it have two entries, one for `1` and one for `True`? Or does something else happen?

Example

```
d = {}
d[1] = "one"
d[True] = "yes"
print(d)
```

Output:

```
{1: 'yes'}
```

Only one entry exists. The key printed is `1` (not `True`), but the value is `"yes"` — the value from the *second* assignment. That's the part that trips people up: the key identity from the first insertion

survives, but the value gets overwritten by the second insertion.

Intuition

The instinct most developers have is to think about keys the way they think about variable names — as distinct labels. `1` "looks like" an integer and `True` "looks like" a boolean, so surely they're different keys, right?

But a dictionary doesn't compare keys by what type they are. It compares them by two things: their hash value, and their equality. Python's rule is that objects which compare equal must produce the same hash — this is a hard requirement of the language, not an implementation detail. So the real question isn't "are `1` and `True` different types?" It's "does `1 == True` evaluate to `True`?"

In Python, `bool` is literally a subclass of `int`. `True` is `1`, and `False` is `0`, at the value level. So:

```
>>> 1 == True
True
>>> hash(1) == hash(True)
True
```

Since they're equal and hash equal, a dictionary treats them as the *same key*. When you insert `d[1] = "one"`, Python creates a slot for that hash/equality bucket, remembers the key object `1`, and stores `"one"`. When you then insert `d[True] = "yes"`, Python computes the hash, finds that slot already occupied by an equal key, and does what dictionaries always do on a duplicate key: it keeps the original key object and updates the value in place. That's why the printed key stays `1` but the value becomes `"yes"`.

This is the same mental model you'd use to reason about any hash table — the value part is mutable per key, but the key identity, once inserted, doesn't get replaced by a later "equal" key.

Brute Force Solution

There isn't really a "brute force vs optimal" algorithm here — this is a language-semantics question, not an optimization problem. But it's useful to think about the brute-force way you'd verify this behavior without any dictionary tricks: manually track insertions in a list of `(key, value)` pairs, and only add a new pair if no existing pair's key compares equal to the new key.

```
def brute_force_set(pairs, key, value):
    for i, (k, v) in enumerate(pairs):
        if k == key:
```

```
            pairs[i] = (k, value)  # keep original key, update value
            return
    pairs.append((key, value))
```

Advantages: dead simple, easy to reason about, no need to understand hashing at all.
Disadvantages: this is O(n) per insertion because you scan every existing pair to check for equality. A real dictionary does not do this.

Time complexity: O(n) per insert, O(n²) for n insertions. **Space complexity:** O(n).

Optimal Solution

Python's dict is a hash table under the hood, so lookups and insertions are O(1) on average. The optimal "solution" here is really just understanding the mechanism CPython uses:

Step	What happens
`d[1] = "one"`	`hash(1)` is computed, an empty slot is found in the internal table, `(1, "one")` is stored there
`d[True] = "yes"`	`hash(True)` is computed — same value as `hash(1)` — table probes that slot, finds an entry whose key (`1`) compares equal to `True` , and **updates the value only**
`print(d)`	Iterates stored entries; shows the *original* key object (`1`) mapped to the *latest* value (`"yes"`)

The key insight: a dict never asks "is this the exact same object?" — it asks "does this hash to the same bucket, and does it compare equal?" Both conditions are satisfied by `1` and `True` , so as far as the dictionary is concerned, they are the same key, full stop.

Time complexity: O(1) average per operation (hash table). **Space complexity:** O(n) for n unique keys.

Python Code

```
def demonstrate_dict_collision() -> dict:
    """Show that 1 and True collide as dictionary keys."""
    d = {}
    d[1] = "one"
    d[True] = "yes"
    return d


if __name__ == "__main__":
    result = demonstrate_dict_collision()
    print(result)          # {1: 'yes'}
```

```
print(len(result))     # 1
print(1 in result)     # True
print(True in result)  # True
```

Code Walkthrough

- `d = {}` creates an empty hash table with no entries.
- `d[1] = "one"` computes `hash(1)`, finds an open slot, and stores the key object `1` alongside `"one"`.
- `d[True] = "yes"` computes `hash(True)`. Since `hash(True) == hash(1)`, Python probes that slot, checks `True == 1` (which is `True`), and concludes this is the same key. It overwrites the stored value with `"yes"` but leaves the stored key object (`1`) untouched.
- `return d` gives back a dict with exactly one entry.
- The `__main__` block confirms the behavior three ways: printing the dict, checking its length, and checking membership for both `1` and `True`.

Dry Run

Starting state: `d = {}`

1. `d[1] = "one"` → table: `{1: "one"}`
2. `d[True] = "yes"` → hash matches existing slot for `1`; equality check `True == 1` passes → value replaced → table: `{1: "yes"}`
3. `print(d)` → `{1: 'yes'}`
4. `len(d)` → `1`
5. `True in d` → `True`, because membership testing uses the same hash+equality rule

Complexity Analysis

- **Time:** O(1) average for each `dict` insertion and lookup, because CPython's dict is implemented as an open-addressed hash table. This is correct because hashing an `int` / `bool` is O(1), and probing a nearly-empty table resolves in a constant number of steps on average.
- **Space:** O(n) for n unique keys stored, since `1` and `True` collapse into a single stored entry rather than two.

Alternative Solutions

If you actually wanted two independent entries for "1" and "True" conceptually, you'd need to break the equality tie — for example, by using string keys (`"1"` and `"True"`) or a tuple that includes the type (`(int, 1)` vs `(bool, True)`):

```
d = {}
d[(int, 1)] = "one"
d[(bool, True)] = "yes"
print(d)  # {(int, 1): 'one', (bool, True): 'yes'}
```

This works because tuples compare element-wise, and `int != bool` as types even though `1 == True` as values. It's a useful pattern any time you need to key on both a value and its type.

Edge Cases

- **Empty dict:** inserting into `{}` always succeeds; no collision possible on the first key.
- **`0` and `False`:** same collision as `1` / `True`, since `hash(0) == hash(False)` and `0 == False`.
- **Floats that equal ints:** `1.0 == 1` is `True` and `hash(1.0) == hash(1)`, so `d[1.0]` and `d[1]` also collide.
- **Custom objects with overridden `__eq__` / `__hash__`:** the same rule applies — if two objects compare equal and hash equal, they collapse into one key, regardless of type.
- **Objects that are equal but hash differently:** this violates Python's data model and leads to undefined/broken dict behavior — always keep `__eq__` and `__hash__` consistent on custom classes.

Common Mistakes

1. Assuming dict keys are compared by type instead of by value and hash.
2. Assuming the *last* inserted key object replaces the *first* — it doesn't; only the value updates, the original key stays.
3. Overriding `__eq__` on a custom class without also overriding `__hash__`, breaking dict behavior entirely.
4. Using mutable objects (like lists) as dict keys, which isn't just a style issue — it's disallowed because hashes must stay stable.
5. Forgetting that `bool` is a subclass of `int` in Python, leading to unexpected collisions in real code (e.g., using flags as dict keys alongside integer IDs).

Interview Questions

- Why must equal objects have equal hashes in Python?
- What happens if you override `__eq__` but not `__hash__` on a class?
- How does CPython resolve hash collisions internally (open addressing vs chaining)?
- Why is `bool` a subclass of `int` in Python's type hierarchy?
- Can you use a tuple as a dictionary key? What about a list?

Similar Problems

- **LeetCode 1. Two Sum** — relies on hashing values to indices in O(1) lookups.
- **LeetCode 49. Group Anagrams** — uses a dict keyed by a canonical form (sorted string or tuple) to bucket collisions intentionally.
- **LeetCode 205. Isomorphic Strings** — depends on correct key equality semantics between two dicts.
- **LeetCode 242. Valid Anagram** — another hash-counting problem where key equality determines correctness.

Key Takeaways

Python dictionaries key on hash **and** equality, never on type. Because `bool` is a subclass of `int` and `True == 1`, they collide as dict keys — the second insertion overwrites the value but keeps the first key object. This isn't a quirk to memorize in isolation; it's a direct consequence of Python's data model, and understanding it makes you better at predicting dict behavior in real production code.

Watch the Video

Prefer to see it explained live? Watch the full breakdown here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that breaks down the kind of Python and data-structure details that look small but reveal how the language really works under the hood — the same details that come up in coding interviews and in real bugs.

Call To Action

If this cleared something up, drop a like on the video and subscribe on YouTube for more of these deep dives. For longer breakdowns and interview prep delivered straight to your inbox, subscribe to the Substack newsletter. Got a Python gotcha you want explained next? Drop it in the comments, and share this with a dev who's still surprised by it.

Quick Review

Python dict collisions: when you see 'same hash, different key', think open addressing probing.

CPython dicts use open addressing, not chaining; colliding keys probe alternate slots via perturbation.

Time complexity of dict get/set under heavy collisions?

Average O(1), but worst case degrades to O(n) if many keys collide into the same probe sequence.

Space complexity of a Python dict with n entries?

O(n), though CPython over-allocates the table (resizes at ~2/3 load factor) for fewer collisions.

Trickiest bug when relying on dict key hashing?

Using mutable or badly-implemented __hash__/__eq__ causes unequal objects to collide or equal objects to hash differently.

Dict collision handling vs Java HashMap's approach?

Python probes to a new slot (open addressing); Java chains colliding entries in a linked list/tree per bucket.

Interview follow-up: why does dict order look insertion-preserved despite collisions?

Since 3.7, a separate dense array tracks insertion order; the hash table only stores indices into it.
